DroneRS Model / ML Team

Two-person beginner onboarding packet

Version 1.0 — August 25, 2026

Your mission: understand, measure, improve, quantize, and export the person-following model while keeping its embedded interface stable. In this project, "backend" means data, PyTorch, training, evaluation, quantization, and DORY export.

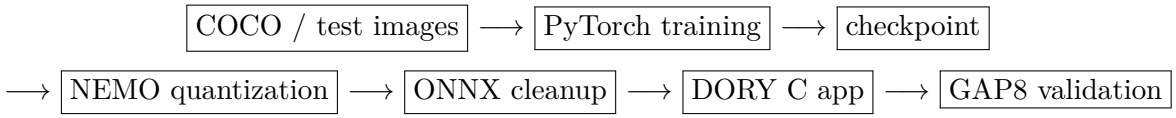
Do not start by training a new network. First reproduce inference from the bundled checkpoint and create a small, understandable baseline report. Training and quantization come after you can explain what the current model sees and where it fails.

1. What Already Works

Repository	`https://github.com/DavidLiu2/pytorch-ssd`
Baseline branch	`unstable`
Baseline commit	`b11e9da`
Model	`plain_follow` with `xbin9_size_bucket4` head
Checkpoint	`artifacts/plain_follow_best_follow_score.pth`
Input	$1 \times 128 \times 128$ grayscale image
Output	14 values: x position, visibility, and apparent size
Deployment status	Generated GAP8 app passes GVSOC, all 9 layers exact
Missing from Git	COCO images, local evaluation packs, full training logs

The production checkpoint and GAP8 app are baselines. Never overwrite them during an experiment. Write all student outputs under `logs/` or `student_projects/` and use a new Git branch.

2. The ML-to-Drone Pipeline



The firmware team consumes the final input/output contract. If your model change modifies shape, preprocessing, ordering, scale, or threshold, it is an interface change and must be reviewed with them before merging.

Vocabulary

Model	A function with learned parameters that maps an image to numbers.
Checkpoint	Saved model parameters plus metadata.
Inference	Running a trained model without changing its parameters.
Training	Updating parameters to reduce a loss on labeled examples.
Loss	A number that tells the optimizer how wrong a prediction was.
Metric	A measurement used to understand quality; it is not optimized directly.
Quantization	Representing operations with small integers so they run efficiently on GAP8.
ONNX	A portable graph representation used between PyTorch/NEMO and DORY.
DORY	Code generator that converts the cleaned graph into a GAP8 C application.

3. Team Responsibilities

Person B1	Data, training, evaluation	Own sample labels, preprocessing, metrics, training experiments, and failure analysis.
Person B2	Quantization and export	Own checkpoint metadata, NEMO/ONNX/DORY flow, numerical parity, and GVSOC release evidence.

Both people must be able to run ordinary PyTorch inference. B2 should not become the only person who understands export, and B1 should not change the output head without B2 and the firmware team.

4. Install This Tech Stack

Shared base tools

Use Windows 10/11 with WSL 2 Ubuntu, Visual Studio Code, Git, and Docker Desktop. This matches the validated workflow.

In Administrator PowerShell:

```
wsl --install
wsl --update
wsl -l -v
```

Install VS Code for Windows and its WSL and Python extensions. Install Docker Desktop, enable its WSL 2 engine, and enable integration for Ubuntu.

Official setup references:

- WSL: https://learn.microsoft.com/en-us/windows/wsl/install
- VS Code WSL: https://code.visualstudio.com/docs/remote/wsl
- Docker WSL: https://docs.docker.com/desktop/features/wsl/
- Python virtual environments: https://docs.python.org/3/tutorial/venv.html
- PyTorch local setup: https://pytorch.org/get-started/locally/

In Ubuntu:

```
sudo apt update
sudo apt install -y git python3 python3-venv python3-pip build-essential
```

```
git --version
python3 --version
```

Clone the project

Use a path without spaces and clone with the historical directory name expected by some scripts:

```
mkdir -p /mnt/c/DroneRS
cd /mnt/c/DroneRS
git clone https://github.com/DavidLiu2/pytorch-ssd.git pytorch_ssd
cd pytorch_ssd
git switch unstable
git pull --ff-only
git rev-parse --short HEAD
```

The commit should be `b11e9da` or newer. If not, contact David.

Week-1 training environment

Keep virtual environments outside the repository:

```
python3 -m venv ~/venvs/droners-train
source ~/venvs/droners-train/bin/activate
python -m pip install --upgrade pip setuptools wheel
python -m pip install -r requirements_trainenv.txt
python -c "import torch; print(torch.__version__); print(torch.cuda.is_available())"
```

CPU mode is expected and sufficient for week 1. Do not install random newer versions when a pinned requirements file exists.

Later: quantization/export environments

The repository intentionally separates environments:

`requirements_trainenv.txt`	Ordinary CPU PyTorch work. Install this first.
`requirements_nemoenv.txt`	NEMO quantization and ONNX export. Use a separate environment.
`requirements_doryenv.txt`	DORY graph/code generation and validation helpers. Heavy and version-sensitive.
Docker image	Pinned Bitcraze AI-deck/GAP8 SDK used for GVSOC build and execution.

Set up NEMO/DORY in a scheduled team session using Python 3.10 or 3.11 and the pinned files. Do not spend the first week independently debugging TensorFlow, ONNX, or compiler compatibility. A shared known-good environment or prebuilt development image is preferred.

5. Verify the Baseline

With any Python 3 interpreter, verify the packaged artifacts:

```
python3 tools/verify_plain_follow_handoff.py
```

Expected: `plain_follow handoff integrity check: PASS`.

With the training environment active, choose a local JPG or PNG and run:

```
python inference_follow_demo.py \
  --ckpt artifacts/plain_follow_best_follow_score.pth \
  --image /absolute/path/to/your_image.jpg \
  --vis-thresh 0.60
```

The script center-crops, converts to grayscale, resizes to 128x128, loads the checkpoint, and prints both raw and decoded values. Use images you have permission to use. Do not commit identifiable photos.

6. Understand the 14-Value Output

The head is `xbin9_size_bucket4`:

Indices 0–8	x-bin logits	Argmax selects one of nine horizontal bins. Centers run from about −0.889 (left) to +0.889 (right).
Index 9	visibility logit	Converted to visibility confidence; the checkpoint selected threshold is 0.60 for normal PyTorch evaluation.
Indices 10–13	size logits	Argmax selects centers 0.125, 0.375, 0.625, or 0.875.

Read `utils/follow_task.py` for the authoritative head specification and decoder. Never copy a new, slightly different decoder into a second file without a good reason.

Useful metrics include visibility precision/recall/F1, no-person false-positive rate, x mean absolute error, size mean absolute error, and the combined follow score. No single number tells the whole story.

7. Initial Project: Baseline Behavior Report

Goal: measure what the shipped checkpoint does on a small, understandable set of images before changing the model. Produce a repeatable script, a CSV, and a short failure-analysis report.

Create:

```
student_projects/backend_baseline/
  run_baseline.py
  sample_labels.example.csv
  test_run_baseline.py
  report.md
  README.md
```

Keep actual photos under `data/onboarding_private/`; `data/` is ignored by Git.

Build a 20-image learning set

Use 10 images with a person and 10 without a person. Include easy and difficult cases: left/center/right, near/far, bright/dim, partial person, clutter, and an object that might resemble a person. Use your own non-sensitive images or approved public images.

For each image, record simple human labels:

- `expected_visible`: yes/no
- `expected_horizontal`: left/center/right when visible
- `expected_size`: near/medium/far when visible

- `notes`: why the example may be difficult

Write one batch-inference script

Reuse model loading and preprocessing from `inference_follow_demo.py`, and use the existing runtime decoder in `utils/follow_task.py`. Load the model once, not once per image. Write `results.csv` with:

- image name and expected labels
- visibility confidence and predicted visible
- predicted x bin/value and size bucket/value
- whether visibility was correct
- a brief error note

Do not build a graphical application. A reliable command-line script and a readable CSV are the goal.

Divide the work

- B1 owns the labeled set, metrics, and failure analysis.
- B2 owns checkpoint inspection, reusable inference code, and a comparison between PyTorch output and the checked-in deployment manifest.
- Both review the CSV and select the three most informative failures.

One controlled experiment

Compare visibility thresholds 0.50 and 0.60 on the same 20 images. Do not train yet. Explain which errors change and why a lower threshold trades fewer missed people for more false positives.

Definition of done

- A new clone can run the script using commands in the README.
- The model is loaded once and inference uses `torch.no_grad()`.
- At least 3 automated tests cover malformed labels, empty folders, and output-column creation.
- The report contains a small results table, three failure examples, and one proposed next experiment.
- No production checkpoint, generated app, or private image is committed.
- The team gives a five-minute demo and opens one reviewed pull request.

8. Second Project: Tiny Training Smoke Test

Do this only after the baseline report and after COCO is available. Download COCO 2017 train images, validation images, and train/validation annotations from:

Official COCO dataset website and downloads

Expected layout:

```
data/coco/images/train2017/
data/coco/images/val2017/
data/coco/annotations/instances_train2017.json
data/coco/annotations/instances_val2017.json
```

Run a deliberately small smoke test from `pytorch_ssd`:

```
python train.py \
  --model-type plain_follow \
  --follow-head-type xbin9_size_bucket4 \
  --epochs 1 --batch_size 4 --num_workers 0 \
  --max-train-batches 10 --max-val-batches 5 \
  --output_dir pytorch_ssd/logs/backend_onboarding_train
```

This proves the loop works; it does not produce a good model. Do not compare a ten-batch smoke checkpoint with the production model as if they were trained equally.

After that, B2 can run the documented release smoke flow in a mentor-led session. Always include `-skip-application-promotion` for an experiment so it cannot replace the verified app.

9. Suggested Two-Week Schedule

Day 1	Install base tools, clone, verify integrity, run one image.
Day 2	Read model factory, inference demo, and output decoder together.
Days 3–4	Create labels, batch script, CSV, and automated tests.
Day 5	Compare thresholds, write failure report, demo, and merge.
Week 2	Download COCO, run tiny training smoke, learn losses/metrics.
End of week 2	Joint contract review with firmware team and plan one controlled model experiment.

10. Git and Experiment Rules

```
git switch unstable
git pull --ff-only
git switch -c backend/<your-name>-baseline
```

Every report must record the commit, checkpoint hash/name, command, data subset, and metric definitions. Change one important variable at a time. A result that cannot be reproduced is a note, not project evidence.

Never commit `data/`, `logs/`, virtual environments, private images, or ad-hoc checkpoints. Do commit code, small example label templates, tests, and concise reports.

11. Copy-Paste AI Starter Prompt

> You are tutoring two first-year college students who are new to Python and machine learning. We are working in the `pytorch-ssd` repository on branch `unstable`, baseline commit `b11e9da`. The current model is `plain_follow`; its checkpoint is `artifacts/plain_follow_best_follow_score.pth`; input is one 128x128 grayscale image; and the head is `xbin9_size_bucket4` with 14 outputs.

Our first project is a Baseline Behavior Report, not a new model. We need a batch-inference script that reuses `inference_follow_demo.py` and `utils/follow_task.py`, loads the model once, evaluates 20 locally labeled images, and writes a CSV. We will compare visibility thresholds 0.50 and 0.60 and describe three failures.

Act as a patient teacher. Define each ML term briefly, work in small steps, and explain why each test matters. Do not change the model architecture, generated `application/`, production checkpoint, or dependency versions. Do not tell us to download random packages; use `requirements_trainenv.txt`. Ask us to paste a relevant file or error instead of guessing. Protect privacy: never ask us to upload private images.

First response only: (1) explain inference, logits, argmax, sigmoid, threshold, and checkpoint in plain English; (2) show the proposed project file tree; (3) propose CSV columns and three automated tests; and (4) give only the first step for reading labels and discovering images. Stop so we can implement it and report back.

12. How to Ask for Help

Provide the exact command, complete error text, Python version, active environment name, `git status -short`, `git rev-parse -short HEAD`, and what you tried. Stop after 30 minutes on the same dependency problem. Do not repeatedly reinstall the environment; ask for a paired setup session.

13. First-Day Checklist

- ☐ I can explain training versus inference.
- ☐ I am working in a virtual environment, not system Python.
- ☐ The handoff integrity check prints **PASS**.
- ☐ I ran the bundled checkpoint on one image.
- ☐ I can identify the x, visibility, and size slices.
- ☐ I chose B1 or B2 and opened my first GitHub issue.
- ☐ I understand why we are measuring the baseline before training.

Source of truth: repository README, checkpoint metadata, `utils/follow_task.py`, validation manifest, and reviewed experiment records. AI output is assistance, not evidence.